



DNS dynamique sur debian 12.5

Légende:

- rouge = nom de commande
- violet = chemins et noms de fichiers
- bleu = lignes des fichiers de configuration

Prérequis :

- Avoir un DHCP opérationnel
- Avoir un DNS configuré

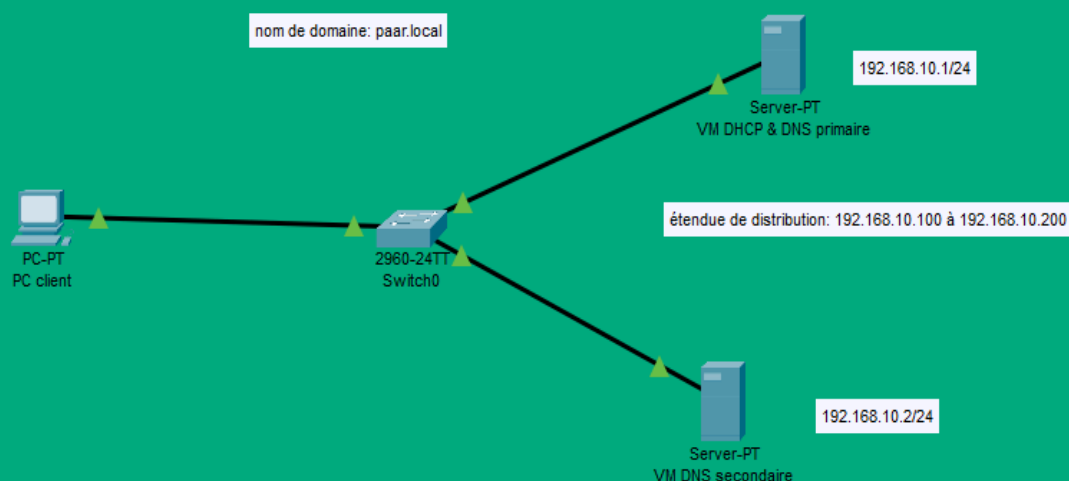
Sommaire:

Objectif.....	1
1.Schéma du réseau local.....	1
2. Modification de la configuration du serveur DNS.....	2
Modification du fichier de conf Bind.....	2
Activer les mises à jour dynamique.....	2
Appliquer les modifications:.....	3
3.Phase de test.....	3
Prérequis avant de tester.....	3
Vérifications dans la conf.....	4
Test des connectivités.....	5

Objectif

Dans ce TP, on veut faire en sorte que le DNS se mette à jour automatiquement grâce à une configuration spécifique du DHCP.

1.Schéma du réseau local



2. Modification de la configuration du serveur DNS

Modification du fichier de conf Bind

- Tout d'abord, on modifie le fichier `etc/bind/named.conf.local` avec `nano`

```
zone "paar.local" IN {
    type master;
    file "db.paar.local";
    notify yes;
    allow-transfer {192.168.10.2;};
    allow-update {127.0.0.1;};
};

zone "10.168.192.in-addr.arpa" IN {
    type master;
    file "db.10.168.192";
    notify yes;
    allow-transfer {192.168.10.2;};
    allow-update {127.0.0.1;};
};
```

- On ajoute les parties `allow-transfer {192.168.10.2}` qui autorise le transfert de données vers le serveur auquel l'adresse ip correspond
- Quand à la partie `allow-update{127.0.0.1}` , elle autorise les mises à jour dynamiques des enregistrements DNS
- Le reste était déjà écrit lors de la configuration du DHCP et du DNS
- Attention à le faire deux fois, pour la zone directe et la zone inversée

Activer les mises à jour dynamique

- On modifie le fichier `/etc/dhcp/dhcpd.conf` avec `nano`. Il faut ajouter ou modifier ces lignes.

```
# The ddns-updates-style parameter controls whether or not the server will
# attempt to do a DNS update when a lease is confirmed. We default to the
# behavior of the version 2 packages ('none', since DHCP v2 didn't
# have support for DDNS.)
ddns-update-style interim;
ddns-updates on;
ignore client-updates;
update-static-leases on;
allow-unknown-clients;
```

- `ddns-updates-style interim`
c'est une méthode dynamique pour la mise à jour
- `ddns-updates on`
autorise de la mise à jour
- `ignore client-updates`
la mise à jour est faite par le serveur DHCP
- `update-static-leases on`
mets à jour même en cas d'IP statiques
- `allow-unknown-clients`
admet aussi les clients inconnus au niveau de l'adresse MAC
- On ajoute ceci en plus à la fin du fichier pour donner précisément quel DNS mettre à jour

```
zone paar.local. { primary 127.0.0.1; }
zone 10.168.192.in-addr.arpa. { primary 127.0.0.1; }
```

```
zone paar.local. {primary 127.0.0.1;}
zone 10.168.192.in-addr.arpa. {primary 127.0.0.1;}
```

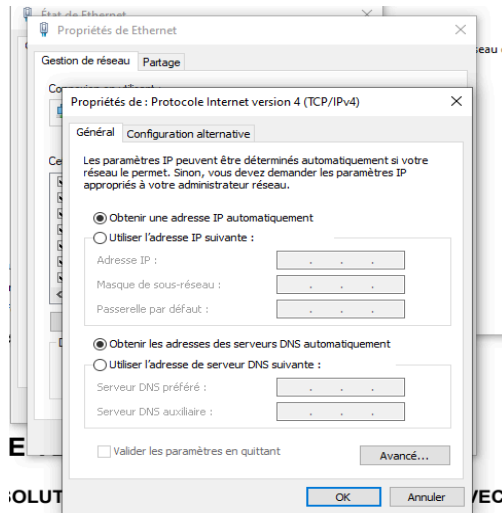
Appliquer les modifications:

Maintenant, on réinitialise avec la commande `systemctl restart bind9` ainsi que la commande `systemctl restart isc-dhcp-server` qui permet d'appliquer les modifications que l'on vient de faire.

3.Phase de test

Prérequis avant de tester

- Passer le PC client en dhcp



- Brancher le PC client au switch

Vérifications dans la conf

- Lorsqu'on va dans `/var/cache/bind` et qu'on exécute `ls -l` on voit bien que c'est automatique car l'extension `jnl` à la fin de plusieurs de ces fichiers montrent qu'ils ne se sont pas créés seuls

```
root@debian-PAAR:~# ls -l /var/cache/bind
total 24
-rw-r--r-- 1 bind bind 626 20 mars 13:42 db.10.168.192
-rw-r--r-- 1 bind bind 2179 20 mars 13:28 db.10.168.192.jnl
-rw-r--r-- 1 bind bind 663 20 mars 13:39 db.paar.local
-rw-r--r-- 1 bind bind 2662 20 mars 13:28 db.paar.local.jnl
-rw-r--r-- 1 bind bind 1421 20 mars 15:49 managed-keys.bind
-rw-r--r-- 1 bind bind 3020 27 mars 13:56 managed-keys.bind.jnl
root@debian-PAAR:~# _
```

- Pour vérifier que la transmission se fait bien on peut exécuter `journalctl -u named` qui permet d'avoir toutes les mises à jour de transfert vers le client dans les logs. l'option `-u` permet de spécifier où faire cette commande

```
client 00x7f7c6778e168 127.0.0.1#14929: updating zone 'paar.local/IN': deleting an RR at C12-23.paar.local A
zone paar.local/IN: sending notifies (serial 14)
client 00x7f7c6778e168 127.0.0.1#14929: updating zone 'paar.local/IN': deleting an RR at C12-23.paar.local TXT
client 00x7f7c6778e168 127.0.0.1#14929: updating zone '10.168.192.in-addr.arpa/IN': deleting rrsset at '102.10.168.192
client 00x7f7c6505d168 192.168.10.2#34017 (paar.local): transfer of 'paar.local/IN': IXFR started (serial 13 -> 15)
client 00x7f7c6505d168 192.168.10.2#34017 (paar.local): transfer of 'paar.local/IN': IXFR ended: 1 messages, 8 record
zone 10.168.192.in-addr.arpa/IN: sending notifies (serial 14)
client 00x7f7c6505b168 192.168.10.2#43149 (10.168.192.in-addr.arpa): transfer of '10.168.192.in-addr.arpa/IN': IXFR s
client 00x7f7c6505b168 192.168.10.2#43149 (10.168.192.in-addr.arpa): transfer of '10.168.192.in-addr.arpa/IN': IXFR e
```

- Si la configuration s'est bien passé, on devrait voir un ajout du PC client et de son adresse ip dans les fichiers de zone directe et inversée sans qu'on ait le besoin d'y toucher:

Zone directe

```
$ORIGIN .
$TTL 86400      ; 1 day
paar.local      IN SOA  Debian-PAAR.paar.local. root.paar.local. (
                    19          ; serial
                    604800      ; refresh (1 week)
                    86400       ; retry (1 day)
                    2419200     ; expire (4 weeks)
                    86400       ; minimum (1 day)
                    )
                    NS          Debian-PAAR.paar.local.
                    NS          debian-DNS-2nd.paar.local.
$ORIGIN paar.local.
$TTL 3600       ; 1 hour
C12-22          A          192.168.10.14
                  TXT       "31920da76a6c4cca2881d5387684c6d8a4"
C12-23          A          192.168.10.114
                  TXT       "318d8cd10a0ed5098265a4ce7b7954df7f"
$TTL 86400      ; 1 day
debian-DNS-2nd  A          192.168.10.2
Debian-PAAR     A          192.168.10.1
poste-coffon    A          192.168.10.53
poste-cosse2    A          192.168.10.50
poste-pascal2   A          192.168.10.51
```

Zone inversée

```
$ORIGIN .
$TTL 86400      ; 1 day
10.168.192.in-addr.arpa IN SOA  Debian-PAAR.paar.local. root.paar.local. (
                    17          ; serial
                    604800      ; refresh (1 week)
                    86400       ; retry (1 day)
                    2419200     ; expire (4 weeks)
                    86400       ; minimum (1 day)
                    )
                    NS          Debian-PAAR.paar.local.
                    NS          debian-DNS-2nd.paar.local.
$ORIGIN 10.168.192.in-addr.arpa.
1              PTR          Debian-PAAR.paar.local.
$TTL 3600       ; 1 hour
114            PTR          C12-23.paar.local.
14             PTR          C12-22.paar.local.
$TTL 86400      ; 1 day
2              PTR          debian-DNS-2nd.paar.local.
50             PTR          poste-cosse2.paar.local.
51             PTR          poste-pascal2.paar.local.
53            PTR          poste-coffon.paar.local.
```

Test des connectivités

Pour tester ici que la nouvelle ligne et donc le PC C12-23 fonctionne bien, on test la connectivité d'abord sur le serveur:

dig C12-23.paar.local

```
root@debian-PAAR:/var/cache/bind# dig C12-23.paar.local

; <<>> DiG 9.18.33-1~deb12u2-Debian <<>> C12-23.paar.local
;; global options: +cmd
;; Got answer:
;; WARNING: .local is reserved for Multicast DNS
;; You are currently testing what happens when an mDNS query is leaked to DNS
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 36365
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 9b0bb6908be544c70100000067dc19a2b7c3ac9ef01d9924 (good)
;; QUESTION SECTION:
;C12-23.paar.local.          IN      A

;; ANSWER SECTION:
C12-23.paar.local.          3600    IN      A      192.168.10.114

;; Query time: 0 msec
;; SERVER: 192.168.10.1#53(192.168.10.1) (UDP)
;; WHEN: Thu Mar 20 14:35:30 CET 2025
;; MSG SIZE rcvd: 90
```

On reçoit une **ANSWER** et l'adresse ip qu'a reçu le PC client grâce au dhcp. Comme ça peut communiquer et que l'adresse IP a été résolue , on en conclut que ça marche bien pour cette partie.

dig -x 192.168.10.114

```
root@debian-PAAR:/var/log# dig -x 192.168.10.114

; <<>> DiG 9.18.33-1~deb12u2-Debian <<>> -x 192.168.10.114
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 41928
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: b8c4e5albf90cf6f0100000067dclad8a9elcf33d780689e (good)
;; QUESTION SECTION:
;114.10.168.192.in-addr.arpa. IN      PTR

;; ANSWER SECTION:
114.10.168.192.in-addr.arpa. 3600 IN      PTR      C12-23.paar.local.

;; Query time: 0 msec
;; SERVER: 192.168.10.1#53(192.168.10.1) (UDP)
;; WHEN: Thu Mar 20 14:40:40 CET 2025
;; MSG SIZE rcvd: 115
```

On reçoit une **ANSWER** et le nom du PC client. Comme ça peut communiquer et que le nom du PC est retrouvé, on en conclut que ça marche bien pour cette partie.

Maintenant, il faut tester les zones directes et inversées mais cette fois dans l'invité de commande du PC client et même chose, il faut que ça communique et que l'adresse ip soit convertie en nom et vice-versa.

nslookup PC12-23.paar.local

```
C:\Users\pgriffon>nslookup C12-23.paar.local
Serveur :   Debian-PAAR.paar.local
Address:  192.168.10.1

DNS request timed out.
    timeout was 2 seconds.
DNS request timed out.
    timeout was 2 seconds.
Nom :      C12-23.paar.local
Address:  192.168.10.114
```

nslookup 192.168.10.114

```
C:\Users\pgriffon>nslookup 192.168.10.114
Serveur :   Debian-PAAR.paar.local
Address:  192.168.10.1

Nom :      C12-23.paar.local
Address:  192.168.10.114
```

Maintenant on fait un dernier test : sur le serveur, un ping client.virtualix.local doit répondre (par contre on ne peut pas vérifier la résolution entre les clients parce qu'il n'y en a qu'un !).

ping C12-23.paar.local

```
root@debian-PAAR:/var/log# ping C12-23.paar.local
PING C12-23.paar.local (192.168.10.114) 56(84) bytes of data.
64 bytes from C12-23.paar.local (192.168.10.114): icmp_seq=1 ttl=128 time=0.236
ms
64 bytes from C12-23.paar.local (192.168.10.114): icmp_seq=2 ttl=128 time=0.401
ms
64 bytes from C12-23.paar.local (192.168.10.114): icmp_seq=3 ttl=128 time=0.687
ms
64 bytes from C12-23.paar.local (192.168.10.114): icmp_seq=4 ttl=128 time=0.663
ms
^C
--- C12-23.paar.local ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3140ms
rtt min/avg/max/mdev = 0.236/0.496/0.687/0.187 ms
```

Ici on peut voir que le serveur et le PC communiquent, ce qui, comme c'est le dernier test possible, nous montre que l'installation du DNS dynamique sur debian 12.5 a bien été réalisé et fonctionne correctement.